Master Thesis Proposal
Prüfer: Prof. Dr. Marius Kloft
Supervisor: Saurabh Varshneya

ROBUST REINFORCEMENT LEARNING AGAINST SPURIOUS CORRELATIONS

May 16th, 2026

Zahra Khodabakhshian
Mtrk.nr.: 426198
Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau (RPTU)
jov98mam@rptu.de

1. Motivation

Reinforcement learning (RL) agents can perform very well in the environment in which they are trained, but this performance does not always transfer to slightly different settings. One possible reason is shortcut learning: the agent may rely on an observed feature that is predictive during training but is not actually necessary for solving the task. This issue is studied in machine learning under related terms such as Clever Hans behavior, spurious correlations, and confounding [1], [2], and recent work has shown that similar failures can also appear in reinforcement learning [3].

This problem is the main motivation for my thesis. In a robotic manipulation task, for example, the color of an object can be correlated with its position. A policy may then use color as an easy cue, even though color is not what makes the lifting behavior succeed. If this relationship changes during evaluation, the learned policy may fail because it has learned a shortcut instead of a robust manipulation strategy. The aim of this work is to study this failure mode in a controlled RL setting and to test whether a robustness-oriented training procedure can make the policy less sensitive to such misleading correlations.

2. Research Direction

This thesis follows the direction opened by the RSC-MDP framework from *Seeing is not Believing* [3], which was published at NeurIPS 2023. I focus on the empirical question of how much a reinforcement learning policy depends on a shortcut when the training environment contains a controlled spurious correlation.

The goal is to build a small but clear simulation-based experimental setting where this behavior can be measured. The data used in the thesis is generated by online interaction with the robosuite environment, which makes it possible to control the training and evaluation distributions precisely. The thesis therefore asks whether a policy trained in a confounded environment still performs well when the spurious relationship is removed or reversed, and whether a robustness-oriented training procedure can improve this behavior compared with standard SAC.

3. Method: RSC-SAC Adaptation

Building on this direction, my proposed method adapts the empirical RSC-SAC idea to my experimental setting. The central problem is that the variable creating the spurious correlation is not something the agent can directly observe or intervene on. Because of that, I do not try to manipulate the hidden confounder itself. Instead, I approximate what such a change could look like by modifying observed states from the replay buffer and using these modified samples during training.

The method can be understood in four steps. First, the agent collects normal SAC experience in the environment. Second, during training, some states from a replay-buffer batch are perturbed using the rule from Eq. 7 of [3]. Third, a learned structural transition model predicts new next-state and reward targets for the perturbed state-action pairs. Finally, SAC is updated on a mixed batch containing both real transitions and generated transitions.

This means that Soft Actor-Critic (SAC) [4] is still the base learning algorithm. The robust part comes from changing the data distribution seen during the SAC update. Ideally, the generated transitions reduce the agent's dependence on the training-time shortcut and encourage the policy to use features that remain useful when the spurious relationship changes.

Training procedure: RSC-SAC style update

Given: policy π, replay buffer D, transition model P_θ, graph parameter φ, modification ratio β

for each training step t **do**

$a_t \leftarrow \pi(\cdot \mid s_t)$

execute a_t and observe (s_{t+1}, r_t)

$D \leftarrow D \cup \{(s_t, a_t, s_{t+1}, r_t)\}$

sample a batch B from D

choose a subset $I \subset B$ with size controlled by β

perturb the selected states using Eq. 7:

$\tilde{s}_j \leftarrow \mathrm{Perturb}(s_j), \quad j \in I$

predict new transition targets:

$(\hat{s}'_j, \hat{r}_j) \leftarrow P_{\theta(\tilde{s}_j, a_j, G_\varphi)}$

replace selected rows by $(\tilde{s}_j, a_j, \hat{s}'_j, \hat{r}_j)$

train P_θ and G_φ with

$L = \|s'_j - \hat{s}'_j\|_2^2 + \|r_j - \hat{r}_j\|_2^2 + \lambda \|G_\varphi\|_p$

update the SAC actor and critics on the mixed batch

end for

3.1. Perturbing Replay-Buffer States

The first RSC-specific step is to generate a perturbed version of the current state. The perturbation rule uses one state dimension i and replaces it by the same dimension from another

sample k in the batch. The chosen sample should be different in dimension i, but still close in the remaining dimensions:

$$s_t^i \leftarrow s_k^i, \quad k = \arg\max \frac{\left\| s_t^i - s_k^i \right\|_2^2}{\sum_{\tilde{i}} \left\| s_t^{\tilde{i}} - s_k^{\tilde{i}} \right\|_2^2}, \quad k \in \{1, ..., K\}$$

In my words, this creates a counterfactual-like state: one part of the observation is changed, while the rest remains close to something that was actually seen in the replay buffer.

3.2. Learning the Structural Transition Model

After perturbing the current state, the original transition target is no longer fully reliable. The next state and reward in the replay buffer came from the original state, not from the perturbed one. For this reason, the method learns a transition model that can generate a new target for the modified state-action pair:

$$(\hat{s}_{t+1}, \hat{r}_t) \leftarrow P_{\theta(\tilde{s}_t, a_t, G_\varphi)}$$

The important idea from the paper is that this model is not just a standard black-box dynamics model. It also learns a sparse graph G_φ between the input variables (s_t, a_t) and the output variables (s_{t+1}, r_t). In my implementation, this graph is represented through differentiable binary-concrete edge samples. The sparsity term encourages the model to use only the dependencies that are useful for prediction, instead of freely connecting every input dimension to every output dimension.

The architecture follows the model described in Appendix D.1 of [3]. Each scalar state or action dimension is encoded with a shared encoder together with a learnable position embedding. The learned graph then mixes these encoded features, and a shared decoder predicts each dimension of the next state and the reward. I use the following schematic to summarize the model:

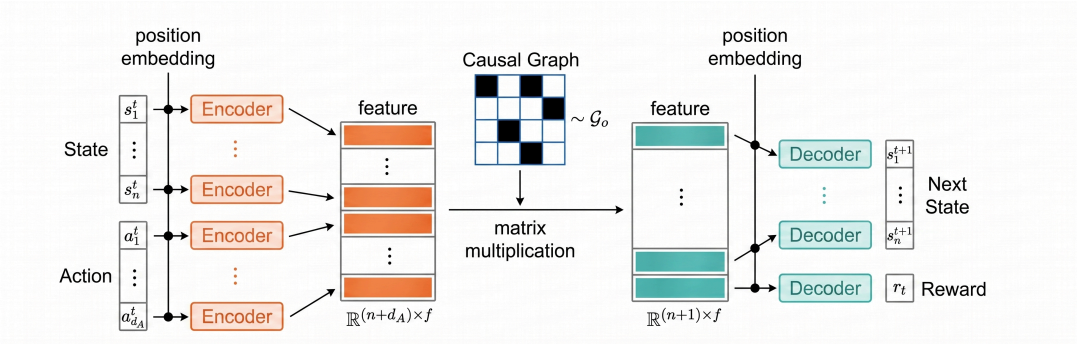

Figure 1: Schematic of the structural transition model used to generate next-state and reward targets for perturbed states.

The model is trained with a prediction loss for the next state and reward, together with a graph sparsity penalty:

$$L(\theta, \varphi) = \left\| s_{t+1} - \hat{s}_{t+1} \right\|_2^2 + \left\| r_t - \hat{r}_t \right\|_2^2 + \lambda \left\| G_\varphi \right\|_p$$

This generated transition is then inserted into the SAC batch. In this way, the policy and critic are trained not only on the original confounded data, but also on transitions where the spurious relationship has been weakened by perturbation.

3.3. Environment and Confounding Design

The experimental environment is based on the robosuite `Lift` task with a Panda robot [5]. The task is to lift a cube from the table. In the original task, cube color is not necessary for solving the manipulation problem: the robot can complete the task using the cube position, gripper state, and other physical state variables. I therefore use cube color as a controlled spurious feature.

To create a shortcut-learning setting, I modify the reset distribution of the environment. At the beginning of each episode, the cube is placed either on the left or on the right side of the workspace, and its color is set to either green or red. During nominal training, these two variables are correlated: for example, left cubes are green and right cubes are red. This makes color predictive of position during training, although color itself is not causally required for lifting.

The agent uses state-based observations rather than image observations. Since cube color would otherwise not be visible to the policy, I append the RGB value of the cube to the flattened robosuite state. The final observation therefore contains the original physical state plus three additional color dimensions `[r, g, b]`. The action space remains the standard 4-dimensional Panda control action.

I define three main environment modes:

`confounded`	Training setting. Cube position and cube color are correlated, e.g. left-green and right-red.
`shifted_indep`	Test setting. Cube position and cube color are sampled independently. This removes the shortcut.
`shifted_swapped`	Test setting. The training mapping is reversed. This makes the shortcut actively misleading.

The shifted environments test whether the learned policy depends on the color-position shortcut rather than making the physical lifting task harder. A robust policy should still lift the cube when color no longer predicts the training position pattern.

4. Research Questions

This thesis investigates the following research questions:

1. Does a standard SAC agent trained in the confounded Lift environment rely on the spurious color-position correlation?
2. Can RSC-style counterfactual transition generation improve robustness under shifted test environments?
3. How important is the perturbation strategy, especially when comparing paper-faithful random perturbation with color-focused perturbation?

5. Thesis Contribution

This thesis makes the following contributions:

- a controlled robosuite `Lift` setup for studying a color-position spurious correlation under distribution shift,
- an RSC-SAC-style implementation with Eq. 7 state perturbations, a structural transition model, and mixed real/synthetic SAC updates,

- **extend RSC-SAC by selecting perturbation candidates that are not only state-similar but also action-invariant, so the swapped state factors are more likely to capture spurious rather than causal features.**
- a comparison between standard SAC and RSC-SAC variants under confounded and shifted evaluation modes,
- an analysis of different perturbation choices, including paper-faithful random perturbation and color-focused perturbation,
- an extension of the implementation to additional robosuite tasks to test whether the same idea transfers beyond the initial `Lift` setup,
- ...

6. Preliminary Results

The following tables show preliminary results from experiments comparing a baseline SAC agent (Base) with an RSC-SAC agent using random perturbation (Random).

Table 1: Testing reward on shifted environments.

Environment	Base (SAC)	Random (RSC-SAC)
Shifted independent	0.412	0.647
Shifted swapped	0.000	0.312

Table 2: Testing reward on nominal environments.

Environment	Base (SAC)	Random (RSC-SAC)
Confounded (nominal)	1.000	0.824

The baseline SAC policy reaches perfect performance in the nominal confounded environment, but its performance drops strongly under shift, especially in the swapped setting. This suggests that the baseline policy does not only learn the physical lifting behavior, but also uses the training-time color-position relationship. The RSC-SAC variant has lower nominal performance, but it improves performance in both shifted settings. This supports the hypothesis that generated counterfactual transitions can reduce dependence on the spurious feature, although more seeds are needed before making a final claim.

References

[1] R. Geirhos *et al.*, "Shortcut Learning in Deep Neural Networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020, doi: 10.1038/s42256-020-00257-z.

[2] D. Steinmann *et al.*, "Navigating Shortcuts, Spurious Correlations, and Confounders: From Origins via Detection to Mitigation," *arXiv preprint arXiv:2412.05152*, 2024.

[3] W. Ding, L. Shi, Y. Chi, and D. Zhao, "Seeing is not Believing: Robust Reinforcement Learning against Spurious Correlation," in *Advances in Neural Information Processing Systems*, 2023.

[4] T. Haarnoja *et al.*, "Soft Actor-Critic Algorithms and Applications," *arXiv preprint arXiv:1812.05905*, 2018.

[5] Y. Zhu *et al.*, "robosuite: A Modular Simulation Framework and Benchmark for Robot Learning," *arXiv preprint arXiv:2009.12293*, 2020.